
SPIKE: An Adaptive Dual Controller Framework for Cost-Efficient Long-Horizon Game Agents

Anonymous Author(s)

Affiliation

Address

email

Abstract

Long-horizon multimodal agents in open-world games must stay goal-directed across many low-level interactions under tight token and latency budgets. Existing approaches often trade off costly per-step reasoning against reactive execution that can drift, repeat failures, and recover poorly. Our key idea is to reuse strategic reasoning across locally stable segments and reinvoke it at event boundaries. We present SPIKE, an **adaptive dual controller framework** for cost-efficient long-horizon game control. Its Strategic Controller performs low-frequency global planning, failure analysis, and recovery, while its Reactive Controller handles fast local execution under a strict token budget. An **Event Trigger** monitors visual change, task progress, repeated actions, and failure signals to decide when control should stay reactive or escalate to strategic reasoning. **Hierarchical Memory** separates short-term experience reuse in the State-Action Memory Bank (SA-MB) from structured evidence in the State-Action Knowledge Graph (SA-KG), allowing each controller to retrieve the context it needs. This design reuses strategic proposals over multiple reactive steps, supports local override when plans become stale, and reserves expensive reasoning for moments where extra deliberation is useful. On the Lite-100 split of STARDOJO, SPIKE improves Lite-100 success rate (SR) by **5.0 percentage points** (38.5% relative) over the strongest Lite-100 baseline and Budgeted SR by **9.3 points** (75.6% relative) over the strongest budgeted baseline. It also reduces token consumption by **54.9%** and latency by **40.8%**. Ablations show that **event triggering**, **reactive override**, and **heterogeneous memory** each contribute to success and recovery, supporting **selective reasoning** rather than reasoning at every step.

1 Introduction

Recent advances in Large Language Models (LLMs), Vision-Language Models (VLMs), and vision-action foundation models have enabled increasingly capable agents for open-world games [17, 29], web navigation [4, 38], and general computer control [16, 25]. Unlike single-step question answering, these environments require agents to act over long, changing interaction histories with delayed feedback. In tasks such as STARDOJO [26], which spans 1,000 tasks and a Lite-100 subset, an agent may need dozens or hundreds of steps to coordinate tool use, navigate, handle menus, switch subgoals, and recover from failures without a privileged game-state API. The central challenge is therefore **closed-loop control under scarce deliberation**: reasoning should be spent on bottlenecks and recovery, while routine low-level actions must stay cheap. This creates a **planning-latency-memory trilemma**: deeper reasoning improves deliberation but raises inference cost, lightweight reactive execution is fast but short-sighted, and naive memory reuse introduces retrieval noise.

Current agents address only parts of this control problem. As illustrated in Figure 1-(a), planning-intensive frameworks such as CRADLE can reason effectively, but repeated calls to large models make

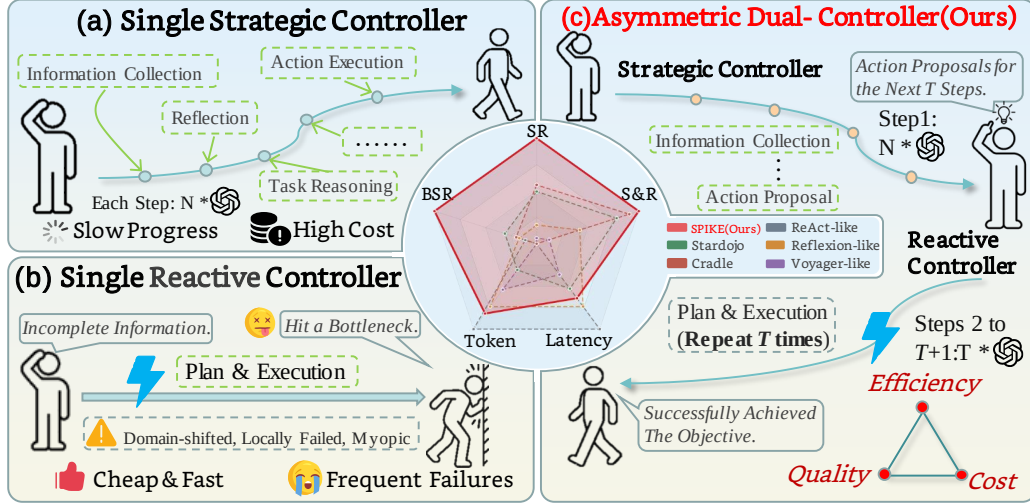


Figure 1: **Motivation for event-triggered deliberation.** Single-Strategic (a) control reasons carefully but is expensive, while Single- Reactive (b) control is cheap but brittle under drift and bottlenecks. (c) SPIKE allocates strategic reasoning only at likely bottlenecks, aiming for higher success and recovery with lower per-step cost.

high-frequency control expensive in both tokens and latency [25]. Conversely, Figure 1-(b) captures reactive agents such as ReAct: they execute cheaply, but become unreliable under environment drift, local execution failure, repetition, and multi-stage task transitions [34]. The desired behavior, shown in Figure 1-(c), is to reuse strategic guidance across routine reactive steps while retaining the ability to replan when the local state changes. Fast-slow and hierarchical agents, including SayCan, Inner Monologue, and SwiftSage [2, 9, 15], as well as ADaPT and Optimus-3 [14, 21], move toward selective reasoning. However, multimodal open worlds still expose three gaps: triggering is often rigid, sudden visual changes are hard to detect from text-only cues, and the fast layer is frequently treated as a passive executor. Meanwhile, long-horizon memory is often stored as a flat prompt buffer or a single retrieval pool, which weakens cross-episode reuse and increases retrieval noise [19, 20].

Motivated by these observations, we propose SPIKE, an **event-triggered deliberation framework** for cost-efficient long-horizon game agents. Our core insight is that strategic reasoning should be amortized across locally stable segments and re-invoked only at event boundaries. Long-horizon game control is temporally uneven: most steps lie in stable local execution, while a few boundaries dominate recoverable failures. This turns reasoning into a resource allocation problem. SPIKE therefore couples three mechanisms: the Event Trigger decides *when* expensive reasoning is worth invoking; the bounded Reactive Controller decides *how* to act safely between deliberations rather than serving as a passive executor; and controller-specific Hierarchical Memory decides *what evidence* supports local action reuse versus strategic replanning. As detailed in Section 2, the memory separates the State-Action Memory Bank (SA-MB) from the State-Action Knowledge Graph (SA-KG), preventing routine execution and replanning from sharing one noisy retrieval context.

Overall, our main contributions are:

- **Event-triggered amortized deliberation.** We formulate long-horizon multimodal control as budgeted allocation of strategic reasoning over event boundaries.
- **Adaptive dual-controller execution.** SPIKE uses strategic planning at escalation points and bounded reactive override during stable local execution.
- **Controller-specific hierarchical memory.** SPIKE separates SA-MB for local state-action reuse from SA-KG for structured strategic evidence.
- **Cost-aware evidence.** On STARDOJO Lite-100 and RDR2, SPIKE improves success–cost trade-offs, with ablations isolating trigger timing, override, and memory structure.

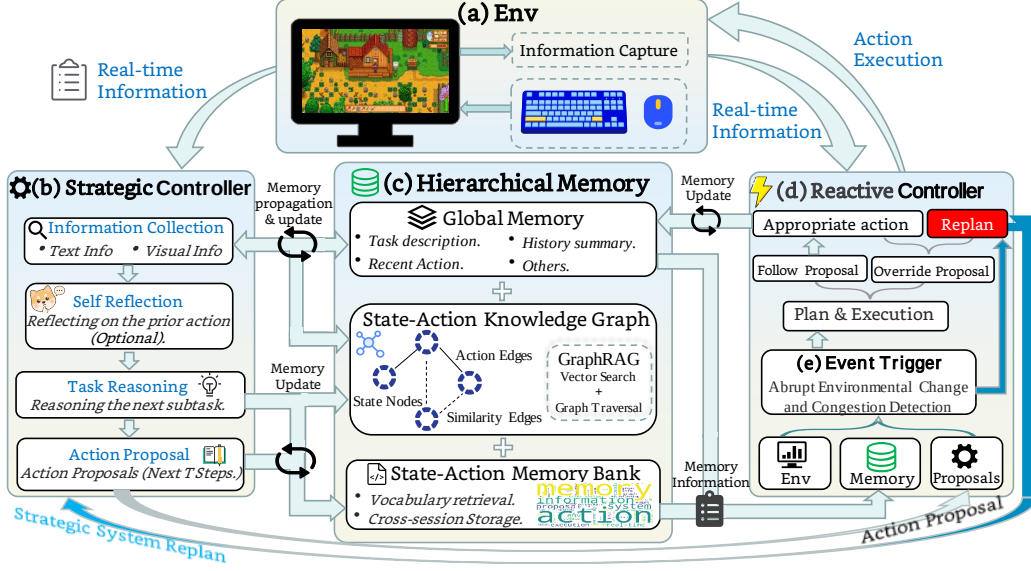


Figure 2: **Closed-loop event-triggered deliberation in SPIKE.** The Strategic Controller performs information collection, optional Self Reflection, task reasoning, and short-horizon action proposal; the Reactive Controller uses real-time signals and retrieved memory to follow, override, or trigger replanning. The Hierarchical Memory module combines Global Memory, the State-Action Memory Bank (SA-MB), and the State-Action Knowledge Graph (SA-KG) through vector search and graph traversal, then propagates updates after action execution.

2 The SPIKE Framework

We model long-horizon open-world control as sequential decision making with multimodal observations, layered memory, recovery, and token-latency budgets: the Event Trigger decides when to deliberate, the Reactive Controller acts between deliberations, and Hierarchical Memory retrieves evidence at routine and escalation boundaries.

Open-world control has a cost-value mismatch: routine steps need fast execution, whereas rare scene changes, stalls, and failures require expensive reasoning. Always-strategic agents spend large-model calls on routine motion; purely reactive agents lack reliable triggers for stale plans. SPIKE treats strategic reasoning as event-triggered, amortizes proposals over stable segments, and retrieves different evidence for local and escalated steps.

2.1 Event-Triggered Dual Controller Framework

For an episode with horizon H , let $o_t = (v_t, u_t, f_t)$ denote the observation at step t , where v_t is the visual screenshot, u_t is textual or user interface (UI) context, and f_t is runner feedback such as progress, position, inventory, or dialogue changes. The agent chooses an executable game action $a_t \in \mathcal{A}$ from history $h_t = (o_1, a_1, \dots, o_t)$. An event-trigger decision $g_t \in \{0, 1\}$ selects the Strategic Controller ($g_t = 1$) or Reactive Controller ($g_t = 0$). For trajectory $\tau = (o_1, a_1, \dots, o_H)$, success is $S(\tau) \in \{0, 1\}$ and the deliberation budget is B . Let $C_t(g_t, h_t)$ denote the history-dependent model-call, token, or latency cost. The target is

$$\max_{\pi_R, \pi_S, g} \mathbb{E}[S(\tau)] \quad \text{s.t.} \quad \mathbb{E} \left[\sum_{t=1}^H C_t(g_t, h_t) \right] \leq B. \quad (1)$$

Inspired by dual-process theories of cognition [11], SPIKE separates slow, high-cost planning from fast local execution. The Strategic Controller handles planning, failure analysis, and task redirection; the Reactive Controller executes local actions under a strict token budget; the Event Trigger and Hierarchical Memory allocate deliberation and evidence. This differs from always-deliberate agents, which call the expensive planner at nearly every step, from fixed fast-slow pipelines, which switch controllers by a static schedule, and from memory-augmented agents with a single retrieval pool. In



Figure 3: **Strategic Controller workflow in one escalated planning cycle.** The example shows how SPIKE collects scene evidence, checks progress, reasons over the task, and emits a short plan for Reactive Controller execution.

92 SPIKE, controller allocation, local override, and memory retrieval are all conditioned on detected
 93 event boundaries, so the framework changes both *when* reasoning is invoked and *what* evidence each
 94 controller sees.

95 As illustrated in Figure 2, SPIKE consists of Env and Information Capture, Strategic Controller,
 96 Hierarchical Memory, Reactive Controller, and Event Trigger. Together they form a closed loop:
 97 capture produces o_t and updates h_t ; the Event Trigger scores whether the current history marks an
 98 event boundary; the selected controller retrieves controller-specific memory and emits a_t ; execution
 99 feedback updates SA-MB, SA-KG, and h_{t+1} ; the next step repeats this trigger-act-update cycle.

100 **Controller roles.** The Strategic Controller is activated when periodic refresh, stuck-state, or
 101 recovery signals require deliberate reasoning. It integrates the current observation, task summary,
 102 recent failure traces, and structured long-term memory, then follows the workflow

Information Collection -> Self Reflection -> Task Reasoning -> Action Proposal.

104 Figure 3 expands the Strategic Controller workflow. The optional Self Reflection stage handles
 105 unclear evidence, contradiction, or repeated failure. The output is a short-horizon proposal for the
 106 next T steps, long enough to guide local execution and compact enough to revise after a scene change.

107 The Reactive Controller handles routine steps under a strict token budget. It uses the latest strategic
 108 proposal, current observation, and Memory Information from SA-MB to emit local actions. When
 109 new evidence invalidates the proposal, it may only apply bounded local corrections such as avoidance,
 110 repositioning, or tool reselection; it cannot change the active subgoal, and failed corrections return
 111 control to the Event Trigger.

112 2.2 Event Trigger as an Online Surrogate for Budgeted Deliberation

113 The Event Trigger operationalizes the budgeted objective as a practical online event-boundary
 114 decision. Ideally, each step would choose whether the expected utility of strategic deliberation
 115 justifies its extra cost:

$$g_t^* = \arg \max_{g \in \{0,1\}} \hat{U}(g | h_t) - \lambda C_t(g, h_t), \quad (2)$$

116 where $\hat{U}(g | h_t)$ estimates progress and recovery value from history and λ controls the cost penalty.
 117 Because the true value of deliberation is not observable online from sparse task success, SPIKE

118 uses a reproducible surrogate event detector. Eq. 3 uses fixed thresholds over visual discontinuity,
 119 progress stagnation, behavioral repetition, and execution failure; these signals identify stale strategic
 120 proposals or unreliable local execution rather than estimating task success. Appendix Sec. F gives the
 121 trigger-act-update loop.

122 The trigger variable $g_t \in \{0, 1\}$ indicates whether step t escalates:

$$g_t = \mathbb{I}[(c_t \geq T) \vee (d_t > \tau_v) \vee (z_t \geq \tau_z) \vee (r_t \geq \tau_r \wedge z_t \geq \tau_{rz}) \vee (\ell_t \geq \tau_\ell)]. \quad (3)$$

123 where c_t counts Reactive steps since the last Strategic call and resets after it, T is the refresh interval,
 124 and $\tau_v, \tau_z, \tau_r, \tau_{rz}, \tau_\ell$ are fixed triggering thresholds. The detector maps signals to observable failure
 125 modes: $d_t = 1 - \cos(\phi(v_t), \phi(v_{t-1}))$ catches stale plans after visual discontinuities, z_t catches
 126 stalled routines, r_t catches behavior loops within the last W actions, and $\ell_t \in \{0, 1, 2, 3\}$ catches
 127 execution failure. We set $\Delta_t = q_t - q_{t-1}$, where q_t is a deterministic progress counter derived from
 128 the shared STARDOJO runner feedback, not a privileged game-state API; $z_t = 0$ when $\Delta_t > 0$ and
 129 $z_t = z_{t-1} + 1$ otherwise. All baselines receive the same raw observation, action schema, execution
 130 feedback, and history fields; SPIKE only adds Event Trigger bookkeeping.

131 The fixed thresholds instantiate the event detector rather than a per-task policy; Table 5 reports
 132 their local sensitivity, and Appendix Sec. F details their computation. These thresholds are tied to
 133 event types rather than individual tasks: within the same game family, where camera motion, action
 134 granularity, and progress feedback have similar scales, the same settings can be reused without task-
 135 by-task replacement. When moving to a different game type, the Event Trigger should be recalibrated
 136 according to the new event statistics, such as visual dynamics, interaction pace, stall duration, and
 137 failure-message density. Failure levels scope recovery: F1 keeps bounded retry local, while F2/F3
 138 escalate invalid actions, stalls, or repeated compound failures when local context becomes unreliable.

139 2.3 Hierarchical Memory

140 Long-horizon tasks risk context overflow, hallucination, and forgetting. Figure 2(c) organizes
 141 Hierarchical Memory around controller-specific roles: Global Memory preserves task continuity,
 142 the State-Action Memory Bank (SA-MB) is a low-latency local action prior for Reactive execution,
 143 and the State-Action Knowledge Graph (SA-KG) is a strategic evidence graph for checking and
 144 replanning rather than a flat retrieval store.

145 Global Memory keeps a fixed-size window of recent observations and actions, together with the
 146 active subgoal, progress indicators, and failure summaries. When the Event Trigger keeps execution
 147 local, the Reactive Controller retrieves nearby state-action traces from SA-MB as local hints, with
 148 the risk of stale habits reduced by recency, reward, and success terms. Given the current query x_t ,
 149 SA-MB ranks item i by

$$S_{\text{MB}}(i \mid x_t) = \rho_i(\alpha_C C_i + \alpha_L L_i + \alpha_R R_i + \alpha_P P_i). \quad (4)$$

150 All terms are normalized before weighting: L_i is the Jaccard overlap between lowercased query and
 151 memory token sets, C_i is token-frequency cosine similarity, $R_i = (\text{clip}(\text{rewardEMA}_i, -1, 1) + 1)/2$
 152 uses an exponential moving average (EMA) of observed rewards, P_i is successes over attempts,
 153 $\rho_i = \exp(-\text{ageHours}_i/24)$, and $\alpha_C, \alpha_L, \alpha_R, \alpha_P$ are fixed retrieval weights. The Reactive Controller
 154 uses the top retrieved summaries as hints rather than executable commands.

155 When the Event Trigger escalates control, the Strategic Controller consults SA-KG for plan checking
 156 rather than local traces alone. SA-KG stores validated transitions as state nodes, action edges, and
 157 similarity edges, inspired by state-action-result graphs in experience-driven agents without privileged
 158 environment APIs [28] but using retrieved graph fragments as evidence rather than directly executing
 159 retrieved actions. Starting from the nearest state nodes for the current subgoal, SA-KG retrieves
 160 graph fragments by vector search, keeps matches above τ_{KG} , and ranks fragment j by

$$S_{\text{KG}}(j \mid x_t) = \beta_C C_j + \beta_P P_j, \quad (5)$$

161 where $C_j = 1 - \text{dist}(x_t, s_j)$ is vector similarity and P_j is the success rate of the best outgoing
 162 action, with execution count used to break ties. Fusion first min-max normalizes S_{MB} and S_{KG}
 163 within the current candidate sets, then uses $S_{\text{fuse}} = \lambda_{\text{MB}} \tilde{S}_{\text{MB}} + \lambda_{\text{KG}} \tilde{S}_{\text{KG}}$; unmatched graph actions
 164 receive zero normalized SA-KG score. After execution, SPIKE writes only successful execution,
 165 interpretable recovery, or planner-validated progress, preventing transient errors from becoming
 166 long-term evidence. Appendix Sec. G gives the storage fields, fusion score, fixed weights, and
 167 promotion rules.

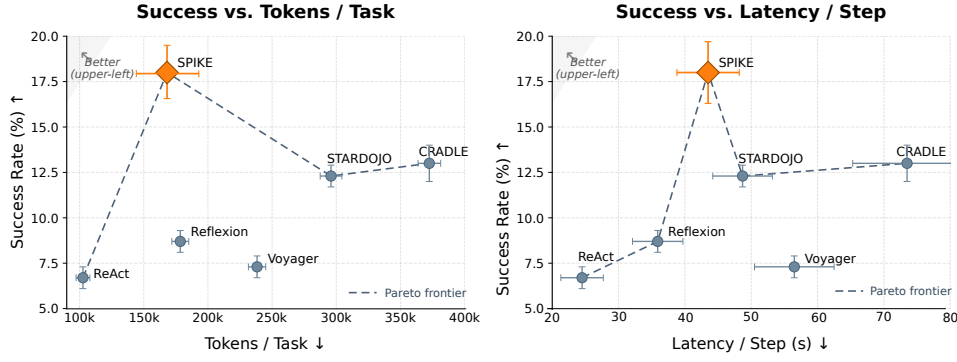


Figure 4: **Effectiveness-efficiency trade-off on Lite-100.** The two panels compare success against token use and latency; upper-left is better. SPIKE stays on the Pareto frontier, improving success without simply spending more compute.

3 Experiments

This section evaluates SPIKE in long-horizon game control. The main Lite-100 study, derived from STARDOJO [26], measures success, LLM-call efficiency, and latency, and recovery, then uses task breakdowns, Pareto analysis, ablations, and qualitative traces to explain the gains. We also test the same adaptive dual controller on Red Dead Redemption 2 (RDR2) to assess transfer across visual domains, action spaces, and task languages.

3.1 Experimental Setup

Our main evaluation uses Lite-100 in STARDOJO [26], which contains 100 tasks across farming, crafting, exploration, combat, and social interaction. Table 1 shows the category and difficulty split. We compare against ReAct-like [34], Reflexion-like [22], Voyager-like [29], CRADLE [25], and the benchmark-native STARDOJO baseline. Unless otherwise specified, all baselines and the default SPIKE use the same Qwen3.5-397B-A17B backend, environment interface, action space, prompt upper bound, decoding settings, and, when applicable, the same frozen visual-change encoder/descriptor. Appendix Sec. C specifies the baseline prompt fields, history truncation, memory isolation, retry handling, and adaptation rules.

Table 1: **Lite-100 task statistics.**

Category	Total	Easy	Medium	Hard
Farming	21	14	3	4
Crafting	14	7	4	3
Exploration	28	15	8	5
Combat	12	3	6	3
Social	25	17	6	2
Total	100	56	23	21

Table 2: **Main comparison on STARDOJO.** “Recovery/Stuck Ratio” is Recovery Rate divided by Stuck Rate. Entries report mean $\pm$ standard deviation over three repeated runs. Table 6 reports the raw stuck and recovery statistics behind this ratio. Bold and underlined entries denote the best and second-best results within each method group; blue shading marks results from our method.

Method	Lite-100 SR $\uparrow$	Budgeted SR $\uparrow$	Tokens / Task (k) $\downarrow$	Latency / Step (s) $\downarrow$	Recovery/Stuck Ratio $\uparrow$
ReAct-like [34]	6.7 $\pm$ 0.6%	9.6 $\pm$ 0.6%	102.6$\pm$5.4	24.5$\pm$3.2	1.61 $\pm$ 0.11
Reflexion-like [22]	8.7 $\pm$ 1.0%	10.4 $\pm$ 0.6%	178.5 $\pm$ 6.6	35.9 $\pm$ 3.8	2.02 $\pm$ 0.14
Voyager-like [29]	7.3 $\pm$ 0.6%	8.1 $\pm$ 1.0%	238.3 $\pm$ 6.7	56.5 $\pm$ 6.0	1.73 $\pm$ 0.15
CRADLE [25]	13.0 $\pm$ 1.7%	10.7 $\pm$ 0.6%	372.5 $\pm$ 8.8	73.5 $\pm$ 8.2	2.49 $\pm$ 0.18
STARDOJO [26]	12.3 $\pm$ 1.4%	12.3 $\pm$ 1.4%	295.9 $\pm$ 8.4	48.7 $\pm$ 4.5	2.37 $\pm$ 0.18
SPIKE (Default Qwen3.5-397B)	18.0$\pm$1.7%	21.6$\pm$1.7%	168.1 $\pm$ 5.7	43.5 $\pm$ 4.7	2.75$\pm$0.12
SPIKE (GPT-5.4)	20.3$\pm$1.7%	23.7$\pm$1.4%	107.8$\pm$2.1	27.9$\pm$4.3	3.50$\pm$0.29
SPIKE (Gemini-3.1-pro)	17.7$\pm$1.4%	20.3$\pm$1.4%	134.3$\pm$3.2	33.8$\pm$3.3	2.66$\pm$0.19

We report Lite-100 success rate (SR), Budgeted SR, tokens per task, latency per step, and Recovery/Stuck Ratio. Lite-100 SR follows the benchmark step-cap protocol, whereas Budgeted SR uses equal difficulty-dependent LLM-call budgets (120/200/600 calls for easy/medium/hard tasks), where one call denotes one large-model invocation. Because this protocol constrains LLM calls rather than environment steps, Budgeted SR should be read as an efficiency metric rather than a stricter subset of Lite-100 SR. Unless otherwise specified, all quantitative results report mean and standard deviation over three repeated runs. Appendix Sec. B defines call counting, and Table 6 reports the raw events behind the Recovery/Stuck Ratio.

3.2 Main Results on Lite-100

Under the same backend and environment interface, Table 2 shows that SPIKE reaches the **highest Lite-100 SR** at 18.0% and the **highest Budgeted SR** at 21.6%. Compared with CRADLE, the strongest Lite-100 baseline, SPIKE improves success by 5.0 points while reducing tokens from 372.5k to 168.1k and latency from 73.5 s to 43.5 s. It also improves Budgeted SR by 9.3 points over the strongest budgeted baseline, STARDOJO. The absolute success rate remains modest, but the result shows a better success-cost trade-off rather than a solved benchmark.

Figure 4 and Table 3 show that this gain **does not come from uniformly spending more compute** or from one task family: SPIKE stays on the Pareto frontier and has the highest mean success rate across all task families. Combat and Social remain the hardest categories because moving enemies and long inter-house routes stress navigation and planning; Appendix Sec. H details these failure modes. The GPT-5.4/Gemini-3.1-pro rows reach 20.3/17.7% SR with 107.8/134.3k tokens per task; Appendix Sec. B explains why we treat them as controlled auxiliary backend runs rather than isolated model-quality comparisons.

Table 3: **Task-type success breakdown on Lite-100.** Each column reports the success rate for one task family. Entries report mean $\pm$ standard deviation over three repeated runs.

Model	Farming $\uparrow$	Crafting $\uparrow$	Exploration $\uparrow$	Combat $\uparrow$	Social $\uparrow$	Total $\uparrow$
ReAct-like [34]	14.3 $\pm$ 0.0%	9.5 $\pm$ 4.1%	4.8 $\pm$ 2.1%	0.0 $\pm$ 0.0%	4.0 $\pm$ 0.0%	6.7 $\pm$ 0.6%
Reflexion-like [22]	17.5 $\pm$ 2.7%	11.9 $\pm$ 4.1%	4.8 $\pm$ 2.1%	2.8 $\pm$ 2.8%	6.7 $\pm$ 2.3%	8.7 $\pm$ 0.6%
Voyager-like [29]	14.3 $\pm$ 0.0%	9.5 $\pm$ 4.1%	6.0 $\pm$ 2.1%	2.8 $\pm$ 2.8%	4.0 $\pm$ 0.0%	7.3 $\pm$ 0.6%
CRADLE [25]	25.4 $\pm$ 2.7%	16.7 $\pm$ 4.1%	10.7 $\pm$ 3.6%	2.8 $\pm$ 2.8%	8.0 $\pm$ 4.0%	13.0 $\pm$ 1.0%
STARDOJO [26]	20.6 $\pm$ 2.7%	19.0 $\pm$ 4.1%	9.5 $\pm$ 2.1%	5.6 $\pm$ 4.8%	8.0 $\pm$ 0.0%	12.3 $\pm$ 0.6%
SPIKE (Ours)	34.9$\pm$2.7%	23.8$\pm$4.1%	13.1$\pm$2.1%	8.3$\pm$8.3%	10.7$\pm$2.3%	18.0$\pm$1.7%

3.3 Ablation Study

The ablation study separates controller allocation from memory design. In Table 4, *Single Strategic Controller* variants retain deliberate reasoning but pay high cost, reaching at best 16.3% SR with 305.4k tokens per task. *Single Reactive Controller* variants are cheaper but fail more often, reaching only 10.3% SR even with Hierarchical Memory. The full SPIKE improves both sides of this trade-off by letting the Strategic Controller handle escalation points and the Reactive Controller handle routine execution. This pattern suggests that the main benefit is not simply averaging the two controllers, but assigning each controller to the temporal regime where it is most useful: deliberate reasoning at discontinuities and cheap execution during stable local progress.

The component removals show that this is not just a two-controller ensemble. Removing the *Event Trigger* or *reactive override* drops SR to 15.7%, indicating that both escalation timing and local correction are needed. Memory ablations show the other half of the mechanism: replacing Hierarchical Memory with *Global Memory* lowers SR from 18.0% to 12.3%, while removing *SA-MB* or *SA-KG* weakens recent-experience reuse and structured evidence for replanning. Thus, the gain comes from coupling Event Trigger decisions with controller-specific memory, not from adding more prompt context.

Table 4: **Framework-level and component ablations of SPIKE on Lite-100.** “ Δ SR” is the absolute change in Lite-100 SR in percentage points relative to the full SPIKE. Entries report mean $\pm$ standard deviation over three repeated runs.

Variant	Lite-100 SR	Tokens / Task (k)	Latency / Step	Recovery/Stuck Ratio	Δ SR
<i>Framework-level ladder</i>					
SPIKE (Full)	18.0$\pm$1.7%	168.1 $\pm$ 5.7	43.5 $\pm$ 4.7	2.75$\pm$0.12	0.0
Single Strategic Controller + Global Memory	12.7 $\pm$ 1.0%	286.8 $\pm$ 7.3	57.8 $\pm$ 5.4	2.18 $\pm$ 0.09	-5.3
Single Strategic Controller + Hierarchical Memory	16.3 $\pm$ 0.6%	305.4 $\pm$ 6.9	61.1 $\pm$ 5.0	2.32 $\pm$ 0.14	-1.9
Single Reactive Controller + Global Memory	6.7 $\pm$ 0.6%	116.8$\pm$7.3	28.8$\pm$3.4	1.84 $\pm$ 0.05	-11.3
Single Reactive Controller + Hierarchical Memory	10.3 $\pm$ 0.6%	135.4 $\pm$ 6.9	33.1 $\pm$ 3.9	2.05 $\pm$ 0.08	-7.7
Dual Controller + Global Memory	12.3 $\pm$ 1.0%	154.5 $\pm$ 4.9	39.8 $\pm$ 4.1	1.98 $\pm$ 0.12	-5.7
<i>Component removals</i>					
w/o Event Trigger	15.7 $\pm$ 0.6%	198.5 $\pm$ 8.3	39.4 $\pm$ 5.8	2.01 $\pm$ 0.12	-2.3
w/o reactive override	15.7 $\pm$ 0.6%	184.7 $\pm$ 6.5	49.4 $\pm$ 4.8	1.97 $\pm$ 0.08	-2.3
w/o SA-MB	15.0 $\pm$ 1.0%	160.3 $\pm$ 6.1	42.1 $\pm$ 4.6	2.10 $\pm$ 0.12	-3.0
w/o SA-KG	14.3 $\pm$ 0.6%	158.9 $\pm$ 5.2	40.8 $\pm$ 4.4	1.96 $\pm$ 0.12	-3.7

Table 5: Event Trigger threshold sensitivity on Lite-100.

Setting	T	τ_v	Stall trigger	Lite-100 SR	Budgeted SR	Tokens / Task (k)	Strategic calls / step
More strategic	3	0.30	$z_t \geq 3$ or $(r_t \geq 4 \wedge z_t \geq 2)$	17.0%	20.4%	163.3	2.355
Default	4	0.35	$z_t \geq 4$ or $(r_t \geq 5 \wedge z_t \geq 2)$	18.0%	21.6%	168.1	2.273
More reactive	6	0.40	$z_t \geq 5$ or $(r_t \geq 5 \wedge z_t \geq 3)$	15.4%	19.4%	172.5	2.189
No periodic refresh	∞	0.35	$z_t \geq 4$ or $(r_t \geq 5 \wedge z_t \geq 2)$	11.6%	13.8%	173.8	2.024

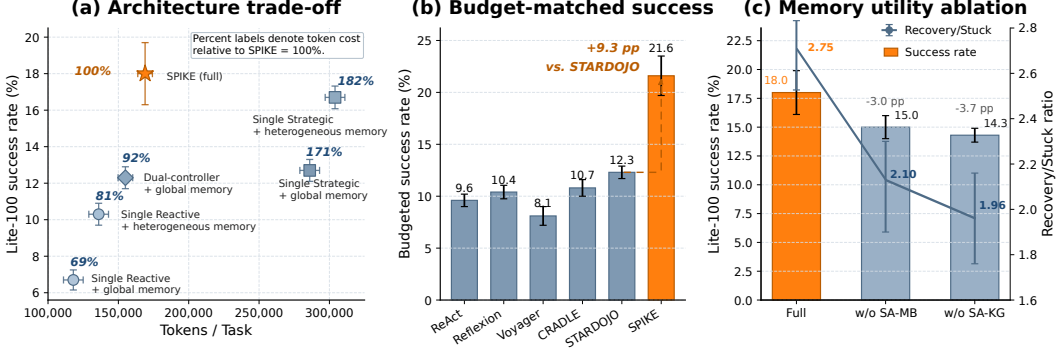


Figure 5: **Mechanistic analysis of SPIKE.** (a) compares controller variants by SR and tokens, showing dual control improves success at lower cost. (b) reports Budgeted SR under difficulty-dependent LLM-call budgets, testing whether event triggering delays budget exhaustion. (c) removes the two memory tracks and reports success and Recovery/Stuck Ratio.

Table 5 tests the local sensitivity of the Event Trigger. The default setting gives the best success–cost balance among the tested variants. Earlier escalation slightly increases Strategic Controller calls but lowers SR, while delayed escalation and removing periodic refresh cause larger success drops. Appendix Sec. F details how the thresholds are computed.

3.4 Mechanistic and Qualitative Analysis

Figure 5 links the gains in success and cost to controller allocation: (a) shows that SPIKE avoids the low-success setting of purely reactive execution and the high-cost setting of always-strategic execution. (b) shows why Budgeted SR improves: the Event Trigger reserves expensive Strategic Controller calls for refresh, stuck, and recovery events, leaving routine steps to the Reactive Controller. (c) connects recovery to memory structure; SA-MB supports fast local reuse, while SA-KG supplies structured evidence for checking and replanning. Together, these results suggest that the Event Trigger improves performance by changing when reasoning is spent, whereas memory improves what evidence the Strategic Controller receives when it is invoked.

Figure 6 illustrates the same mechanism during execution. ReAct-like control keeps issuing low-cost but unproductive actions, because the local loop lacks a strong trigger for revisiting the plan after the visual state changes. An always-strategic controller can reinterpret the scene and recover, but it repeats this expensive reasoning even after the immediate route pattern is understood. SPIKE separates recovery from routine execution: the Event Trigger escalates after the scene change,



Figure 6: **Qualitative goal-reaching comparison.** SPIKE escalates and returns to low-cost local execution once the plan is stable.

Table 6: **Raw stuck and recovery statistics on Lite-100.** “Stuck Events” counts move-blocked and empty-action events from the execution logs. “Recovered Events” counts stuck events after which the agent returns to a normal task-execution state within the next three environment steps. Raw fields are populated from the same runs used for Table 2; auxiliary model rows follow Appendix Sec. B.

Method	Total Steps	Stuck Events	Recovered Events	Stuck Rate ↓	Recovery Rate ↑	Recovery/Stuck Ratio ↑
ReAct-like [34]	5875±31	2115±16	1227±9	36.0%	58.0%	1.61±0.11
Reflexion-like [22]	5820±22	1804±13	1129±8	31.0%	62.6%	2.02±0.14
Voyager-like [29]	5842±46	1957±22	1135±10	33.5%	58.0%	1.73±0.15
CRADLE [25]	5445±64	1497±27	1026±15	27.5%	68.5%	2.49±0.18
STARDOJO [26]	5536±40	1550±15	1029±8	28.0%	66.4%	2.37±0.18
SPIKE (Ours)	5195±43	1434±22	1089±12	27.6%	75.9%	2.75±0.12
SPIKE (GPT-5.4)	4630±25	912±11	629±6	19.7%	69.0%	3.50±0.29
SPIKE (Gemini-3.1-pro)	5215±38	1413±20	1022±14	27.1%	72.3%	2.66±0.19

the Strategic Controller revises the plan with current observation and memory, and the Reactive Controller then handles local clearing. This division keeps high-cost reasoning concentrated around state discontinuities while preserving fast execution for stable local behavior. It explains why Figure 5 shows gains in both success and cost.

3.5 Cross-Benchmark Generalization

We further evaluate transfer on Red Dead Redemption 2 (RDR2), which changes the visual domain, action space, interaction pace, and task language relative to Lite-100. This setting is a stress test for event-triggered deliberation rather than a benchmark-specific tuning exercise: the tasks include navigation, following, search, protection, and combat-like objectives, while photorealistic views and moving characters produce different visual-change and failure statistics. The experiment follows the CRADLE benchmark setup, using the same environment configuration, task interface, action space, and evaluation protocol for all methods. The evaluation contains 13 selected long-horizon tasks, each run three times; Appendix Table A1 lists the full task set. We keep the same adaptive dual controller framework, Event Trigger signal design, and reporting setup, replacing only the game adapter and task set.

Table 7 shows that SPIKE reaches 56.4% success (22/39), compared with 43.6% for CRADLE and 33.3% for the STARDOJO baseline. This is a 12.8 percentage point gain over CRADLE, with 48.5% fewer tokens per step and 47.1% lower latency. The result suggests that the benefit is not limited to Lite-100’s visual style: allocating high-cost reasoning to event boundaries and supporting those calls with Hierarchical Memory also transfers to a more visually dynamic game.

Table 7: **Cross-game generalization on Red Dead Redemption 2 (RDR2).** SR is reported as mean ± standard deviation over three repeated runs on 13 tasks.

Method	SR	Tokens / Step	Latency / Step (s)
ReAct-like [34]	15.4±2.6%	2.1k±0.2k	28.2±2.7
Reflexion-like [22]	17.9±5.1%	3.5k±0.2k	41.4±3.6
Voyager-like [29]	7.7±2.6%	4.8k±0.3k	72.4±3.9
CRADLE [25]	43.6±7.7%	6.6k±0.8k	102.2±8.4
STARDOJO [26]	33.3±5.1%	5.7k±0.5k	67.5±4.1
SPIKE (Ours)	56.4±7.7%	3.4k±0.7k	54.1±4.6

4 Conclusion

This paper shows that long-horizon game agents benefit from coupling structured control with structured memory. **SPIKE** uses adaptive dual control to reserve expensive strategic reasoning for planning and recovery, while leaving routine execution to a lightweight reactive controller. Together with Hierarchical Memory, this design improves success, cost, and recovery on Lite-100 and also transfers to RDR2 under different visual-domain and action-interface assumptions. Ablations further show that **event triggering**, **reactive override**, and **controller-specific memory** are complementary rather than interchangeable parts of the framework.

Limitations. SPIKE can still misroute control when progress is visually subtle, failure signals are noisy, or retrieved memory becomes stale. Its evidence is strongest on Lite-100 and selected RDR2 tasks, so broader game styles, lower-level interfaces, and longer deployments remain important future tests.

References

- [1] Maria Abi Raad, Arun Ahuja, Catarina Barros, Frederic Besse, Andrew Bolt, Adrian Bolton, Bethanie Brownfield, Gavin Buttimore, Max Cant, Sarah Chakera, et al. Scaling instructable agents across many simulated worlds. *arXiv preprint arXiv:2404.10179*, 2024.
- [2] Michael Ahn, Anthony Brohan, Noah Brown, et al. Do as I can, not as I say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691*, 2022.
- [3] Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- [4] Xiang Deng, Yu Gu, Boyuan Zheng, et al. Mind2Web: Towards a generalist agent for the web. *arXiv preprint arXiv:2306.06070*, 2023.
- [5] Darren Edge, Ha Trinh, Newman Cheng, Joshua Bradley, Alex Chao, Apurva Mody, Steven Truitt, Dasha Metropolitansky, Robert Osazuwa Ness, and Jonathan Larson. From local to global: A Graph RAG approach to query-focused summarization. *arXiv preprint arXiv:2404.16130*, 2024.
- [6] Linxi Fan, Guanzhi Wang, Yunfan Jiang, Ajay Mandlekar, Yuncong Yang, Haoyi Zhu, Andrew Tang, De-An Huang, Yuke Zhu, and Anima Anandkumar. MineDojo: Building open-ended embodied agents with internet-scale knowledge. In *Advances in Neural Information Processing Systems*, 2022.
- [7] Shibo Hao, Yi Gu, Haodi Ma, Joshua Jiahua Hong, Zhen Wang, Daisy Zhe Wang, and Zhiting Hu. Reasoning with language model is planning with world model. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023.
- [8] Mengkang Hu, Tianxing Chen, Qiguang Chen, Yao Mu, Wenqi Shao, and Ping Luo. HiAgent: Hierarchical working memory management for solving long-horizon agent tasks with large language model. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics*, pages 32779–32798, 2025. doi: 10.18653/v1/2025.acl-long.1575.
- [9] Wenlong Huang, Fei Xia, Ted Xiao, Harris Chan, Jacky Liang, Pete Florence, Andy Zeng, Jonathan Tompson, Igor Mordatch, Yevgen Chebotar, Pierre Sermanet, Noah Brown, Tomas Jackson, Linda Luu, Sergey Levine, Karol Hausman, and Brian Ichter. Inner monologue: Embodied reasoning through planning with language models. *arXiv preprint arXiv:2207.05608*, 2022.
- [10] Bernal Jimenez Gutierrez, Yiheng Shu, Yu Gu, Michihiro Yasunaga, and Yu Su. HippoRAG: Neurobiologically inspired long-term memory for large language models. In *Advances in Neural Information Processing Systems*, 2024.
- [11] Daniel Kahneman. *Thinking, Fast and Slow*. Farrar, Straus and Giroux, 2011.
- [12] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Kuttler, Mike Lewis, Wen-tau Yih, Tim Rocktaschel, et al. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, 2020.
- [13] Wenhao Li, Wenwu Li, Chuyun Shen, Junjie Sheng, Zixiao Huang, Di Wu, Yun Hua, Wei Yin, Xiangfeng Wang, Hongyuan Zha, and Bo Jin. TextAtari: 100K frames game playing with language agents. *arXiv preprint arXiv:2506.04098*, 2025.
- [14] Zaijing Li, Yuquan Xie, Rui Shao, Gongwei Chen, Weili Guan, Dongmei Jiang, Yaowei Wang, and Liqiang Nie. Optimus-3: Dual-router aligned mixture-of-experts agent with dual-granularity reasoning-aware policy optimization. *arXiv preprint arXiv:2506.10357*, 2025.
- [15] Bill Yuchen Lin, Yicheng Fu, Karina Yang, Faeze Brahman, Shiyu Huang, Chandra Bhagavatula, Prithviraj Ammanabrolu, Yejin Choi, and Xiang Ren. SwiftSage: A generative agent with fast and slow thinking for complex interactive tasks. In *Advances in Neural Information Processing Systems*, 2023.

- [16] Xiao Liu, Hao Yu, Hanchen Zhang, Yifan Xu, Xuanyu Lei, Hanyu Lai, Yu Gu, Hangliang Ding, Kaiwen Men, Kejuan Yang, Shudan Zhang, Xiang Deng, Aohan Zeng, Zhengxiao Du, Chenhui Zhang, Sheng Shen, Tianjun Zhang, Yu Su, Huan Sun, Minlie Huang, Yuxiao Dong, and Jie Tang. AgentBench: Evaluating LLMs as agents. In *International Conference on Learning Representations*, 2024.
- [17] Loic Magne, Anas Awadalla, Guanzhi Wang, Yinzen Xu, Joshua Belofsky, Fengyuan Hu, Joohwan Kim, Ludwig Schmidt, Georgia Gkioxari, Jan Kautz, Yisong Yue, Yejin Choi, Yuke Zhu, and Linxi Fan. NitroGen: An open foundation model for generalist gaming agents. *arXiv preprint arXiv:2601.02427*, 2026.
- [18] Mingyu Ouyang, Siyuan Hu, Kevin Qinghong Lin, Hwee Tou Ng, and Mike Zheng Shou. GameWorld: Towards standardized and verifiable evaluation of multimodal game agents. *arXiv preprint arXiv:2604.07429*, 2026.
- [19] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- [20] Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *ACM Symposium on User Interface Software and Technology*, 2023.
- [21] Archiki Prasad, Alexander Koller, Mareike Hartmann, Peter Clark, Ashish Sabharwal, Mohit Bansal, and Tushar Khot. ADaPT: As-needed decomposition and planning with language models. In *Findings of ACL: NAACL 2024*, pages 4226–4252. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.findings-naacl.264. URL <https://aclanthology.org/2024.findings-naacl.264/>.
- [22] Noah Shinn, Federico Cassano, Ashwin Gopinath, Karthik R. Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems*, 2023.
- [23] Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Cote, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. ALFWorld: Aligning text and embodied environments for interactive learning. In *International Conference on Learning Representations*, 2021.
- [24] Haotian Sun, Yuchen Zhuang, Lingkai Kong, Bo Dai, and Chao Zhang. AdaPlanner: Adaptive planning from feedback with language models. *arXiv preprint arXiv:2305.16653*, 2023.
- [25] Weihao Tan, Wentao Zhang, Xinrun Xu, Haochong Xia, et al. CRADLE: Empowering foundation agents towards general computer control. *arXiv preprint arXiv:2403.03186*, 2024.
- [26] Weihao Tan, Changjiu Jiang, Yu Duan, Mingcong Lei, Jiageng Li, Yitian Hong, Xinrun Wang, and Bo An. StarDojo: Benchmarking open-ended behaviors of agentic multimodal LLMs in production-living simulations with Stardew Valley. *arXiv preprint arXiv:2507.07445*, 2025.
- [27] Weihao Tan, Xiangyang Li, Yunhao Fang, Heyuan Yao, Shi Yan, Hao Luo, Tenglong Ao, Huihui Li, Hongbin Ren, Bairen Yi, Yujia Qin, Bo An, Libin Liu, and Guang Shi. Lumine: An open recipe for building generalist agents in 3D open worlds. *arXiv preprint arXiv:2511.08892*, 2025.
- [28] Chenwei Tang, Lin Long, Xinyu Liu, Jingyu Xing, Zizhou Wang, Joey Tianyi Zhou, Jiawei Du, Liangli Zhen, and Jiancheng Lv. SAG-Agent: Enabling long-horizon reasoning in strategy games via dynamic knowledge graphs. *arXiv preprint arXiv:2510.15259*, 2025. Earlier versions circulated as “Experience-Driven Exploration for Efficient API-Free AI Agents”.
- [29] Guanzhi Wang, Yuqi Xie, Yunfan Jiang, Ajay Mandlekar, Chaowei Xiao, Yuke Zhu, Linxi Fan, and Anima Anandkumar. Voyager: An open-ended embodied agent with large language models. *Transactions on Machine Learning Research*, 2024.
- [30] Zihao Wang, Shaofei Cai, Guanzhou Chen, Anji Liu, Xiaojian Ma, and Yitao Liang. Describe, explain, plan and select: Interactive planning with large language models enables open-world multi-task agents. In *Advances in Neural Information Processing Systems*, 2023.

- 393 [31] Zihao Wang, Shaofei Cai, Anji Liu, Yonggang Jin, Jinbing Hou, Bowei Zhang, Haowei Lin,
394 Zhaofeng He, Zilong Zheng, Yaodong Yang, Xiaojian Ma, and Yitao Liang. JARVIS-1: Open-
395 world multi-task agents with memory-augmented multimodal language models. *arXiv preprint*
396 *arXiv:2311.05997*, 2023.
- 397 [32] Zhe Wu, Yongxin Kang, Dabin Sheng, Junliang Xing, Guokun Wu, Derek Yuen, Donglin Mo,
398 Yuheng Jing, Kai Li, Weilin Luo, Kun Shao, and Yuanchun Shi. ScenDroid: A scenario-level
399 benchmark for long-horizon, time-evolving GUI agents, 2026. URL [https://openreview.](https://openreview.net/forum?id=hBTsLjjw48)
400 [net/forum?id=hBTsLjjw48](https://openreview.net/forum?id=hBTsLjjw48). OpenReview LLA 2026 poster.
- 401 [33] Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L. Griffiths, Yuan Cao, and Karthik
402 Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. In
403 *Advances in Neural Information Processing Systems*, 2023.
- 404 [34] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan
405 Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference*
406 *on Learning Representations*, 2023.
- 407 [35] Zheyuan Zhang, Zehao Wen, Alvin Zhang, Andrew Wang, Jianwen Xie, Daniel Khashabi, and
408 Tianmin Shu. AgentOdyssey: Open-ended long-horizon text game generation for test-time
409 continual learning agents, 2026. URL <https://agentodyssey.github.io/>. Project page.
- 410 [36] Xiangxi Zheng, Linjie Li, Zhengyuan Yang, Ping Yu, Alex Jinpeng Wang, Rui Yan, Yuan
411 Yao, and Lijuan Wang. V-MAGE: A game evaluation framework for assessing vision-centric
412 capabilities in multimodal large language models. *arXiv preprint arXiv:2504.06148*, 2025.
- 413 [37] Andy Zhou, Kai Yan, Michal Shlapentokh-Rothman, Haohan Wang, and Yu-Xiong Wang.
414 Language agent tree search unifies reasoning, acting, and planning in language models. In *Pro-*
415 *ceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings*
416 *of Machine Learning Research*, pages 62138–62160, 2024.
- 417 [38] Shuyan Zhou, Frank F. Xu, Hao Zhu, Xuhui Zhou, et al. WebArena: A realistic web environment
418 for building autonomous agents. *arXiv preprint arXiv:2307.13854*, 2023.

Appendix

Contents

A	Related Work	14
A.1	Computer Control and Game Environments	14
A.2	LLM/VLM Agents for Computer Tasks and Games	14
A.3	Dual Controller Frameworks and Memory for Long-Horizon Agents	14
B	Metric and Reproducibility Details in STARDOJO	15
B.1	Step-budget SR versus Budgeted SR	15
B.2	Compute Resources	15
B.3	Additional Reproducibility Settings	16
B.4	Auxiliary Model Rows	16
C	Baseline and Protocol Details	16
C.1	Baseline Adaptation	16
C.2	LLM-call Accounting	17
C.3	Recovery-event Counting	17
C.4	RDR2 Protocol and Task List	17
C.5	Demo Visualizations	17
D	Controller Prompt Skeletons	19
D.1	Condensed Prompt Skeletons	19
D.2	Concrete Prompt Excerpts	19
E	Hyperparameter Summary	21
E.1	Hyperparameter Table	21
F	Event Trigger Implementation Details	21
F.1	Signal Definitions	21
F.2	Component-ablation Interpretation	22
G	Hierarchical Memory Implementation Details	22
G.1	Memory Ranking and Fusion	22
G.2	Assets and Licenses	23
H	Additional Limitations	23
H.1	Task-family Disparities	23
H.2	Representative Combat Failure Case	23
I	Responsible Use and LLM Disclosure	24
I.1	Ethics Statement	24
I.2	Reproducibility Statement	24
I.3	Broader Social Impact	24
I.4	LLM Disclosure	24

A Related Work

A.1 Computer Control and Game Environments

Benchmarks for agents built on foundation models mainly come from two lines. The first studies software and general computer control, including web, mobile, desktop, and real computer tasks [16, 25, 38]. Recent benchmarks such as ScenDroid and GameWorld/V-MAGE further sharpen this line [18, 32, 36]. These settings are useful for tool use and interface grounding, but they only partly capture the navigation, scene changes, and recovery pressure of dynamic open-world games. ALFWorld aligns text interaction with embodied household environments, while MineDojo provides an open-ended Minecraft benchmark with internet-scale knowledge [6, 23]. Voyager shows that large models can support long-running exploration in Minecraft-like worlds [29]. TextAtari extends this direction to 100K-frame decision-making and makes the trade-off between expensive reasoning and reactive execution explicit [13]. STARDOJO [26] lies between these traditions: it keeps open-world dynamics but requires control without a privileged game-state API, using screen-derived observations and a shared executable action interface, making low-level reliability and control scheduling central.

A.2 LLM/VLM Agents for Computer Tasks and Games

LLM and VLM agents have made strong progress on multi-step computer interaction. ReAct established the observe-reason-act loop [34]. Later agents extended this idea to web navigation, app use, and general computer control [4, 25, 38]. In games, Voyager demonstrates strong large-model planning, while TextAtari shows how long-horizon tasks expose the tension between costly reasoning and reactive execution [13, 29]. DEPS, Lumine, and SIMA extend this line with open-world control [1, 27, 30], while JARVIS-1 adds memory-augmented multimodal control [31].

Scope of comparison. Recent open-world game-agent systems and benchmarks are important context but are not always direct same-table baselines for SPIKE. NitroGen is a vision-action foundation model trained with large-scale behavior cloning on 40,000 hours of gameplay videos across more than 1,000 games [17], whereas SPIKE studies test-time controller allocation without additional agent training. Lumine targets real-time 3D open-world control with an end-to-end VLM agent that maps raw pixels to high-frequency keyboard–mouse actions [27], while STARDOJO Lite-100 uses benchmark tasks, shared runner feedback, and a fixed executable action interface. GameWorld standardizes verifiable evaluation across 34 browser games and 170 tasks, and V-MAGE provides a vision-centric assessment protocol [18, 36]. Because these works differ in training assumptions, environment suites, action interfaces, and verification signals, we discuss them as related positioning rather than direct baselines.

However, many game agents still rely on privileged observations, task scaffolding, or in-environment training. SPIKE instead focuses on a setting with no training and no privileged game-state API: the Reactive Controller must remain responsive under screen-grounded interaction, while strategic reasoning, used only when needed, handles drift, recovery, and task-level redirection.

A.3 Dual Controller Frameworks and Memory for Long-Horizon Agents

Fast-slow and hierarchical control manage the trade-off between costly reasoning and real-time execution. SayCan combines language planning with affordances, Inner Monologue uses closed-loop language feedback, and SwiftSage explores fast-slow cooperation [2, 9, 15]. ADaPT and AdaPlanner further study on-demand decomposition and feedback-adaptive planning [21, 24]. Search methods, including Tree of Thoughts, RAP, and LATS, show the value of deliberate exploration [7, 33, 37]. Reflexion further highlights feedback and retry for recovery [22]. Many prior agents remain planning-intensive or rely on fixed triggering. In contrast, SPIKE uses multimodal event triggering, local autonomous override, and graded recovery so that strategic reasoning is called only when needed.

Memory is equally important for long-horizon behavior. Generative Agents, Reflexion, and MemGPT show that retrieval and reflection can help agents go beyond a single prompt window [19, 20, 22]. HiAgent and Mem0 further study hierarchical working memory and scalable long-term memory [3, 8]. Retrieval-Augmented Generation (RAG), GraphRAG, and HippoRAG show how external stores and graph retrieval can extend model context [5, 10, 12]. Recent agents without privileged environment APIs also use state-action graphs for exploration and delayed reward planning [28]. AgentOdyssey

studies open-ended text games with test-time continual learning and highlights exploration, episodic memory, and planning cost [35]. Our memory design follows this direction but separates roles: SA-MB supports fast local reuse, whereas SA-KG provides evidence for strategic checking and replanning.

B Metric and Reproducibility Details in STARDOJO

For reproducibility, this appendix fixes the task split, evaluation budgets, model backend, LLM-call accounting rule, visual encoder access, and the run-level logging needed to reconstruct each reported metric. All quantitative results in the main paper report mean and standard deviation over three repeated runs. Both Lite-100 SR and Budgeted SR are evaluated on the STARDOJO Lite-100 benchmark split, which contains 100 tasks covering farming, crafting, exploration, combat, and social interaction, with a difficulty split of 56 easy, 23 medium, and 21 hard tasks. Lite-100 SR is the original STARDOJO success rate under the benchmark step caps. Budgeted SR keeps the same tasks and difficulty split, but replaces the step cap with the equivalent large-model call budget derived from the benchmark-native STARDOJO protocol.

Specifically, the STARDOJO baseline invokes the large model four times per environment step, and Lite-100 uses step caps of 30, 50, and 150 for easy, medium, and hard tasks. Multiplying these step caps by four gives the Budgeted SR call budgets of 120, 200, and 600 calls for the three difficulty levels. One LLM call means one invocation of the Qwen3.5-397B-A17B backend that returns one model completion, whether it is made by the Strategic Controller or the Reactive Controller. Calls used for reflection, reasoning, action proposal, reactive action selection, memory querying, retrieval-query summarization, retry, or replanning are all counted separately. Visual encoding, deterministic vector or index lookup, Event Trigger rules, and environment stepping are not counted as LLM calls because they do not return a model completion. Token usage is reported separately as tokens per task.

Unless otherwise stated, both controllers in SPIKE use the same base model, Qwen3.5-397B-A17B. The Strategic Controller uses a careful long-context configuration, whereas the Reactive Controller uses a token-constrained direct-action configuration. All compared agents use the same LLM backend, Qwen3.5-397B-A17B, and share the same environment interface, action space, prompt upper bound, and decoding settings. The same frozen visual encoder is available to methods that include a visual change or visual retrieval component; methods without such a component do not call it. Because Budgeted SR changes the stopping rule from a fixed environment step cap to a fixed large-model call budget, methods that use fewer large-model calls per step can execute more environment actions before exhausting the budget. Budgeted SR can therefore be either higher or lower than standard Lite-100 SR, and should be read as an efficiency metric under equal LLM call budgets rather than a stricter subset of the standard evaluation.

Step-budget SR versus Budgeted SR. The two SR metrics answer different questions. Lite-100 SR follows the original STARDOJO step budget and measures completion under fixed interaction opportunities, but it does not penalize agents that spend more LLM calls inside each step and can favor high-call-per-step methods. Budgeted SR fixes the large-model call budget instead, making LLM-call efficiency visible, but it has the opposite bias: low-call methods may receive more environment steps before their budget is exhausted. We therefore report both metrics together with token usage and latency. A baseline’s lower Budgeted SR can reflect early exhaustion of the artificial call budget rather than an inherent inability to solve the task.

Compute resources. The shared Qwen3.5-397B-A17B backend is served on two eight-GPU MI308X nodes, while CPU workers handle environment execution, logging, memory indexing, and result aggregation. Local environment execution, artifact checks, and aggregation were run on a workstation with 64 GB system memory and an NVIDIA RTX 4060 Ti GPU with 16 GB memory. Experiments are run with eight concurrent rollouts. Each reported number is calculated from three repeated Lite-100 runs. We report end-to-end wall-clock latency per step in the main tables because this metric includes event triggering, memory retrieval, model inference, action-return overhead, and possible communication delay between environment workers and the model-serving nodes. Such transmission delay may affect the measured latency, although all methods use the same serving and concurrency setup. The final reported runs will be accompanied by the corresponding worker configuration, memory budget, and run-time logs.

Additional reproducibility settings. The Lite-100 runner uses eight parallel workers by default and sets `FIXED_SEED=true` and `PYTHONHASHSEED=42`; the framework default fixed seed is 42. Unless overridden by an environment variable, the default decoding temperature is 1.0; fixed-seed Strategic Controller runs use temperature 0.0, and the Reactive Controller’s direct-action request uses temperature 0.1 with thinking disabled. Stage-specific output limits are 1024 tokens for image or state description, 800 for task inference, 512 for self-reflection, and 2048 for action planning; the Reactive Controller uses a 1024-token maximum. LLM-side image inputs are capped at 960-pixel resolution. The recent-history window is five steps. The local image-change encoder is `local-image-embedding-v1` with 1024 dimensions. The State-Action Memory Bank (SA-MB) quick-path retrieval threshold is 0.85, the execution threshold is 0.92, and the returned SA-MB hint count is one. The State-Action Knowledge Graph (SA-KG) uses BAAI/bge-base-en-v1.5 embeddings with 768 dimensions, ChromaDB storage, a maximum of 10,000 entries, $\text{top-}k = 5$, a similarity threshold of 0.85, and write filtering that requires reflection success with confidence at least 0.7.

The local image-change encoder is a deterministic visual descriptor rather than a pretrained external checkpoint. Each observation is EXIF-normalized, converted to grayscale, resized to 32×32 with bilinear interpolation, flattened to 1024 dimensions, scaled to $[0, 1]$, mean-centered, and L2-normalized before cosine distance is computed. This encoder is used only for visual change detection; it is separate from the SA-KG text embedding model BAAI/bge-base-en-v1.5.

Auxiliary model rows. The GPT-5.4 and Gemini-3.1-pro rows in Table 2 are reported as controlled auxiliary backend runs: they use the same Lite-100 split, call budgets, SPIKE configuration, and evaluation protocol while swapping the large-model backend. Tokens per task still depend on model verbosity, retry behavior, recovery frequency, and Event Trigger calls, so the gap between GPT-5.4 and Gemini-3.1-pro reflects the full execution trace rather than model quality alone. Their latency is further affected by serving infrastructure: these closed-source backends are accessed through commercial model-serving APIs, whereas Qwen3.5-397B-A17B is self-hosted on our MI308X nodes, so faster calls can partly reflect provider-side serving optimization.

C Baseline and Protocol Details

This section spells out the shared evaluation interface used for all compared agents. The goal is to isolate controller allocation and memory design rather than differences in backend access, action parsing, retry policy, or hidden environment state. All methods therefore receive the same task text, screenshot or image-derived observations, executable action schema, recent execution feedback, and bounded history summary. Method-specific additions such as reflection, memory, or skill reuse are allowed only when they are part of that method’s intended decision pattern, and they are built from the method’s own rollout history.

Baseline adaptation. All baselines are evaluated through the same STARDOJO runner, action space, task success evaluator, maximum prompt budget, decoding settings, and Qwen3.5-397B-A17B backend used by SPIKE. We keep each baseline’s core decision pattern while standardizing only the input/output interface needed for Lite-100 execution. Their prompts are filled from the same runtime fields whenever applicable: task description, current observation or image-derived facts, executable action schema, skill library, previous action and execution feedback, and the current history summary. The current task, observation, action schema, and latest feedback are kept as required fields; older history and method-specific memory fields are bounded by the same recent-history window of five steps and the same maximum prompt budget.

ReAct-like keeps a reason-act loop over the shared action API and uses only the rolling interaction history. Reflexion-like adds verbal failure reflection and feeds the reflection back into later prompts from its own rollout history. Voyager-like keeps a memory/skill reuse interface, but stores only skills or summaries produced in the STARDOJO runs and executes the same primitive actions as other methods. CRADLE keeps its perception-planning-execution workflow under the shared backend and environment interface. The benchmark-native STARDOJO row uses the original four-call-per-step protocol from STARDOJO under the same runner and evaluation script. LLM transport retries, short-response retries, and parser or format retries use the shared runner configuration; any repeated large-model invocation is counted by the Budgeted SR call counter. Method-specific memories or reflections are isolated and are built only from each method’s own rollout history.

The shared runner validates every proposed action against the same executable schema before dispatch. Invalid or unparsable outputs trigger the same retry path across methods and are still counted as large-model completions when they require another model request. Episode termination is also shared: a rollout ends when the benchmark success evaluator accepts the task, the task reaches its configured step or call budget, or the runner encounters a terminal environment failure. These rules prevent a baseline from gaining extra interaction time through parser differences or uncounted recovery calls.

LLM-call accounting. Budgeted SR counts every large-model completion request made by an agent, including first-pass calls, transport retries, short-response retries, parser or format retries, reflection calls, task-reasoning calls, action-proposal calls, reactive action-selection calls, memory-query calls, retrieval-query summarization calls, retry calls, and replanning calls. Any LLM completion invoked for summarization, reflection, reasoning, memory querying, or action selection is counted. Visual encoding, deterministic vector or index lookup, Event Trigger rules, and environment stepping remain outside the LLM-call counter because they do not return a model completion.

Recovery-event counting. Let N_{step} be the number of environment steps, N_{stuck} the number of move-blocked or empty-action events recorded in the execution log, and N_{rec} the number of such events followed by a return to normal task execution within the next three environment steps. We report

$$\text{StuckRate} = N_{\text{stuck}}/N_{\text{step}}, \quad \text{RecoveryRate} = N_{\text{rec}}/\max(1, N_{\text{stuck}}), \quad (\text{A1})$$

and the Recovery/Stuck Ratio is $\text{RecoveryRate}/\max(\text{StuckRate}, \epsilon)$ with a small ϵ only to avoid division by zero. Table 6 reports the raw fields behind this metric. Rates and ratios are computed for each run and then averaged across the three repeated runs; the reported uncertainty is the sample standard deviation across those runs. To keep the raw event table internally consistent with the reported mean rates and mean total steps, event-count means are rounded from the corresponding mean rate and mean step values, while the event-count standard deviations are empirical across runs. The ratio is intended to diagnose recovery behavior conditional on stuck events. If $N_{\text{stuck}} = 0$, the run has no recovery opportunity and should be reported as “No stuck events” rather than interpreted as a zero Recovery/Stuck Ratio.

RDR2 protocol. The RDR2 generalization experiment follows the CRADLE benchmark setup: all methods use the same environment configuration, task interface, action space, and success-evaluation protocol. The experiment uses 13 tasks, each run three times, for 39 total rollouts. We keep the same controller allocation and Event Trigger signal design as in the Lite-100 evaluation while replacing only the game environment adapter and task set. For each repeated run, we compute the aggregate success rate over the 13 tasks, then report the mean and sample standard deviation across the three repeated runs; the standard deviation is not computed over the pooled 39 task-level rollouts.

RDR2 task list. Table A1 lists the 13 tasks used in the RDR2 generalization experiment. Each task is run three times, yielding 39 total rollouts.

Table A1: **RDR2 task list for cross-benchmark generalization.**

#	Task	#	Task
1	Follow Dutch	8	Lead Horse
2	Go to Town	9	Follow Javier
3	Hitch Horse	10	Search John
4	Protect Dutch	11	Keep Wolves Away
5	Search for Supplies	12	Kill Wolves
6	Go to Barn	13	Return to Camp
7	Search Barn		

Demo visualizations. Figures A1 and A2 provide qualitative examples from the STARDOJO and RDR2 settings, respectively. Figure A2 shows transfer to a visually different 3D setting, where search is decomposed into room entry, object inspection, and local interaction rather than relying on STARDOJO-specific screen patterns. Figure A1 shows the complementary STARDOJO behavior:

652 SPIKE keeps progress stable on a tool-use task while reserving Strategic Controller calls for stalls or
653 scene changes.

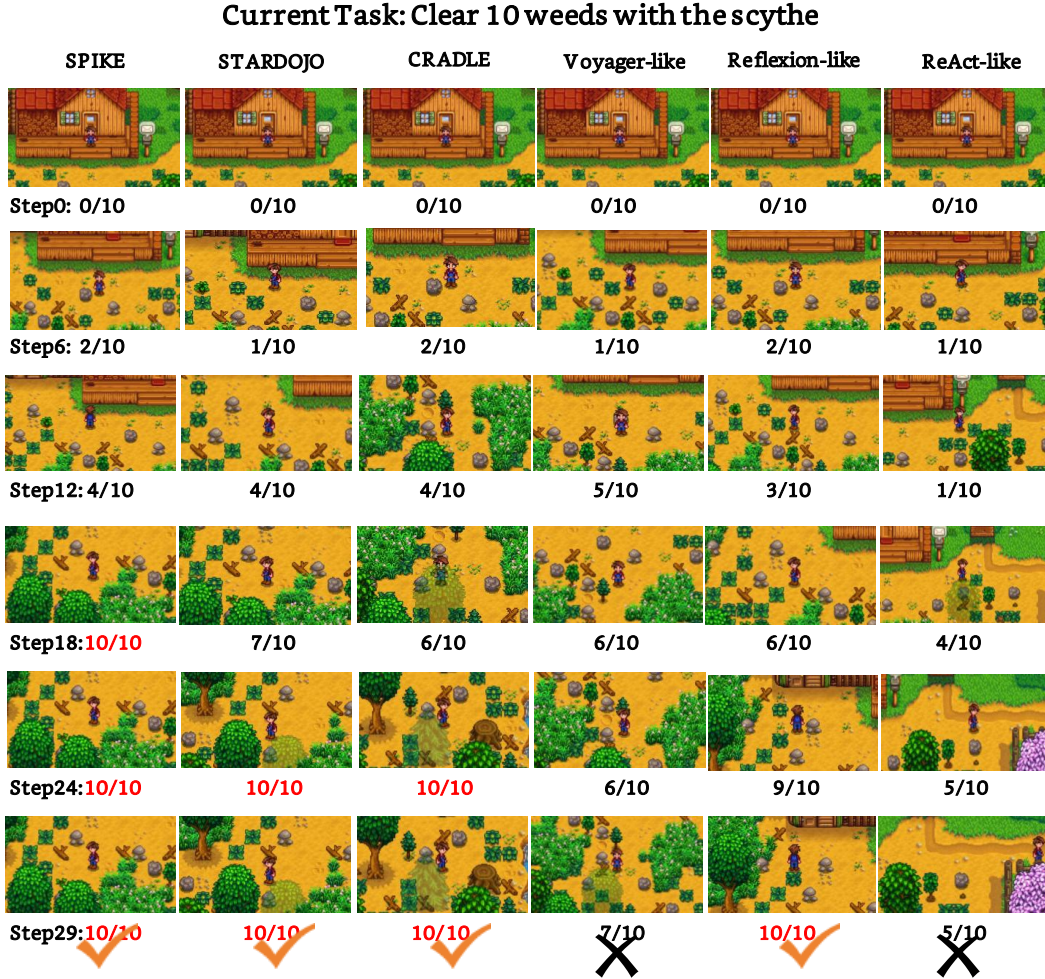


Figure A1: **STARDOJO demonstration example.** The task is to clear 10 weeds with the scythe. The annotated progress counters show how each agent advances over time; SPIKE reaches the target earlier and maintains stable progress under the same visual-control interface.

Current Task: Search for Supplies



Enter the door and observe the surroundings.



Search the cabinet on the left after entering the door



Keep moving to observe the surroundings and search the closet.



Search the nightstand for supplies.

Figure A2: **RDR2 demonstration example.** The task is to search for supplies. The annotations mark the high-level plan sequence: enter the building, inspect the room, search the cabinet and closet, and check the nightstand, illustrating transfer to a visually different navigation and search setting.

D Controller Prompt Skeletons

Table A2 summarizes the prompt skeletons used by the main controller stages; task-specific templates use the same fields with domain-specific constraints such as shopping, cultivation, combat, or navigation. The table is a condensed excerpt rather than a full prompt dump: runtime fields such as observations, SA-MB hints, SA-KG evidence, controller state, and output schemas are filled by the runner, and the complete templates will be released with the implementation.

Table A2: **Condensed prompt skeletons for the controller stages.**

Stage	Main inputs	Required output
Information collection	Screenshot description, on-screen text, UI state, item status, location, dialogue, and environment facts.	Categorized state summary used by both controllers.
Self Reflection	Current task/subtask, previous action and reasoning, current and previous toolbar state, recent history, visual context, and environment facts.	Action-effect judgment, likely failure cause, next-step suggestion, and subtask/task completion status.
Strategic Controller	Task, subtask, acquisition target or route hint, game/UI state, surroundings, blocker summaries, recent feedback, memory evidence, and valid action set.	Short reasoning plus a bounded next-action proposal for the Reactive Controller.
Reactive Controller	Latest Strategic Controller proposal, current observation, SA-MB memory hints, execution feedback, recent action log, and valid action set.	One grounded executable action in the shared action API, or escalation only when no grounded action is available.

Concrete prompt excerpts. The following boxes show shortened excerpts from the actual prompt templates; placeholders are filled at runtime from the current game state, retrieved memory, and execution logs. They are organized to preserve the closed-loop contract used by SPIKE: the Strategic Controller proposes a bounded local intent, the Reactive Controller grounds that intent in the latest

664 observation and may only make local corrections, and Self Reflection converts execution feedback
 665 into progress or failure signals for the next Event Trigger decision.

SPIKE Strategic Controller Prompt Template

666 You are a helpful AI assistant integrated with ‘Stardew Valley’ on the PC, equipped to handle various tasks in the game.
 Here is some helpful information to help you make the decision.
 Current task: <\$task_description\$>
 Current subtask: <\$subtask_description\$>
 Current acquisition target item: <\$target_item\$>
 Location / time / season / health / energy / money: <\$location\$>, <\$time\$>, <\$season\$>, <\$health\$>, <\$energy\$>, <\$money\$>.
 Current position and facing: <\$current_position\$>, <\$facing_direction\$>.
 Surroundings, front-tile summary, blocker signature, nearest grounded target, inventory, toolbar, buildings, NPCs, exits, recent feedback, and valid action set are then provided.
 Event Trigger state: <\$trigger_reason\$>, <\$zero_progress_streak\$>, <\$repeat_tail\$>, <\$failure_level\$>.
 SA-KG evidence for plan checking: <\$retrieved_state_action_graph\$>.
 Output format: brief situation assessment; next local subgoal;
 candidate actions from the valid action set;
 stop condition for returning control to the Event Trigger.

SPIKE Reactive Controller Prompt Template

667 You are a helpful AI assistant integrated with ‘Stardew Valley’ on the PC, serving as the execution brain. The planning brain has already suggested an action. Your job is to compare that suggestion with the latest actual game state and choose the single best action to execute right now.
 Current task: <\$task_description\$>
 Current subtask: <\$subtask_description\$>
 Current observation: location / time / season / health / energy / money: <\$location\$>, <\$time\$>, <\$season\$>, <\$health\$>, <\$energy\$>, <\$money\$>.
 Position, facing direction, menu state, inventory, selected item, surroundings, front-tile summary, toolbar, and latest visual context: <\$current_position\$>, <\$facing_direction\$>, <\$current_menu\$>, <\$inventory\$>, <\$chosen_item\$>, <\$surroundings\$>, <\$front_tile_summary\$>, <\$toolbar_information\$>, <\$image_introduction\$>.
 SA-MB memory hints and recent state summary: <\$history_summary\$>
 Execution feedback: <\$action_feedback\$>
 Recent execution feedback: <\$recent_execution_feedback\$>
 Planning brain’s suggested action: <\$suggested_action\$>
 Reason: <\$suggested_reason\$>
 Execution history in current cycle: <\$execution_log\$>
 Valid action set: <\$skill_library\$>
 If the suggestion matches current facts, adopt it.
 Otherwise choose a better grounded action, but only as a bounded local correction such as facing adjustment, obstacle avoidance, tool reselection, or one-step repositioning.
 Do not change the current subgoal. If no grounded local action can advance the subgoal, request escalation instead of inventing a new plan.
 Output exactly one executable action or an escalation request.

SPIKE Self Reflection Prompt Template

668 Assume you are a helpful AI assistant integrated with ‘Stardew Valley’ on the PC. Your task is to examine the inputs, interpret the in-game context, and determine whether the executed action took effect.
 Current task: <\$task_description\$>
 Current subtask: <\$subtask_description\$>
 Last executed action: <\$previous_action\$>
 Previous reasoning: <\$previous_reasoning\$>
 History summary: <\$history_summary\$>
 Before/after observations, toolbar state, selected item, inventory delta, menu state, position/facing change, and execution message: <\$state_transition_summary\$>.
 Answer whether the action succeeded.
 Identify the most probable failure cause.
 Suggest the next step.
 Decide whether the subtask or task is completed.
 Return a compact status record: success, progress evidence, failure level,
 and whether the Event Trigger should escalate.

669 E Hyperparameter Summary

Table A3: Fixed hyperparameters used in the reported experiments.

Component	Parameter	Value
STARDOJO protocol	Easy/medium/hard step caps	30 / 50 / 150
Budgeted SR	Easy/medium/hard LLM-call budgets	120 / 200 / 600
Evaluation	Repeated runs per task	3
Event Trigger	Periodic strategic refresh T	4
Event Trigger	Action-history window W	5
Event Trigger	Visual-change threshold τ_v	0.35
Event Trigger	Zero-progress threshold τ_z	4
Event Trigger	Repetition / repeated-stall thresholds τ_r/τ_{rz}	5 / 2
Event Trigger	Failure threshold τ_ℓ	2
SA-MB	Semantic weights α_C/α_L	0.55 / 0.45
SA-MB	Reward / success weights α_R/α_P	0.15 / 0.10
SA-MB	Reward exponential moving average (EMA) update η	0.3
SA-MB	Recency decay	24 hours
SA-KG	Embedding model	BAAI/bge-base-en-v1.5
SA-KG	Retrieval threshold / top- k τ_{KG}/k_{KG}	0.85 / 5
SA-KG	Graph weights β_C/β_P	0.6 / 0.4
Fusion retrieval	Weights $\lambda_{MB}/\lambda_{KG}$	0.75 / 0.25

670 F Event Trigger Implementation Details

671 **Event-triggered amortized deliberation algorithm.** Algorithm A3 summarizes the online loop
 672 used by SPIKE. It is written as a reproducibility skeleton: the concrete threshold values are listed in
 673 Table A3, and the signal definitions are expanded below.

Algorithm A1: Event-triggered amortized deliberation in SPIKE
Input: task x , thresholds Θ , SA-MB, SA-KG, budget B
Initialize: proposal $p \leftarrow \emptyset$, history $h_0 \leftarrow \emptyset$
for $t = 1, \dots, H$ **do**
 Observe $o_t = (v_t, u_t, f_t)$ and update history context.
 Compute visual change d_t , stall streak z_t , repetition tail r_t , and failure level ℓ_t .
 Set $g_t \leftarrow \text{EventTrigger}(d_t, z_t, r_t, \ell_t, c_t; \Theta)$.
if $g_t = 1$ **then** retrieve SA-KG/global evidence and update proposal p with the Strategic Controller.
else retrieve local hints from SA-MB.
 Emit a_t with the Reactive Controller using o_t , p , and retrieved hints.
 Execute a_t , receive feedback, and write memory only on success, interpretable recovery, or planner-validated progress.
end for

Figure A3: Reproducibility skeleton for the SPIKE control loop.

674 The fixed thresholds below are not tuned per task; they instantiate event-boundary detection under
 675 the shared STARDOJO interface and are held constant across Lite-100 runs unless explicitly varied
 676 in the sensitivity study. Signals instantiate the symbolic thresholds in Eq. 3 with the fixed settings
 677 in Table A3. The Event Trigger records the same-action tail length r_t , the zero-progress streak z_t ,
 678 and the numeric progress $\Delta_t = q_t - q_{t-1}$. In Lite-100, the progress counter uses only shared runner
 679 feedback: task or subgoal completion, position or facing change, selected-item change, inventory
 680 delta, menu/dialogue transition, and confirmed productive execution. Invalid actions, execution
 681 errors, and repeated productive actions without observable state change do not increase q_t and instead
 682 contribute to z_t or ℓ_t . These are the same raw feedback fields exposed by the common wrapper to all
 683 methods; the derived Event Trigger variables are internal to SPIKE. The Failure Detector assigns ℓ_t
 684 from multimodal traces, with structured execution messages used as additional evidence.

For visual change detection, consecutive observations v_{t-1} and v_t are encoded by a frozen visual encoder $\phi(\cdot)$, and the Event Trigger uses cosine distance:

$$d_t = 1 - \frac{\phi(v_t)^\top \phi(v_{t-1})}{\|\phi(v_t)\| \|\phi(v_{t-1})\|}. \quad (\text{A2})$$

If $d_t > \tau_v$, the event is treated as a scene change, such as an unexpected enemy, menu pop-up, or map transition, and control is escalated before locally plausible but globally wrong actions accumulate. Near static visual state plus repetitive actions contributes to stuck detection, but escalation requires weak progress as well as repetition.

The failure levels are defined as follows. $\ell_t = 0$ denotes normal execution. $\ell_t = 1$ denotes an isolated local issue and triggers bounded local retry. $\ell_t = 2$ denotes hard execution errors, invalid actions, or low progress loops and escalates with the recent failure trace as negative evidence. $\ell_t = 3$ denotes repeated hard failures that invalidate the current plan, flush misleading short-term context, and force top-level replanning. These hyperparameters, together with the memory fusion settings, are fixed for all reported Lite-100 experiments and ablations.

Component-ablation interpretation. Table 4 exposes two coupled failure modes. Removing the Event Trigger spreads Strategic Controller calls across routine steps and weakens adaptation at state transitions; removing reactive override leaves local execution errors until the next escalation. The memory removals affect the evidence available at escalation points: SA-MB mainly supports fast reuse of recent interaction fragments, while SA-KG supplies structured state evidence for progress checking and replanning. These patterns support the main-text claim that scheduling and memory design are coupled rather than independent additions.

Threshold sensitivity. Table 5 in the main text tests whether the Event Trigger depends on a single hand-tuned threshold choice. The *More strategic* setting escalates earlier, the *More reactive* setting delays escalation, and *No periodic refresh* removes the periodic refresh trigger while leaving event-driven scene-change, stall, and failure triggers active. The sweep is intended to test the success–cost trend of adaptive allocation rather than to perform exhaustive hyperparameter search. The action-history window remains fixed at $W = 5$ for all Event Trigger sensitivity settings; the more-reactive setting delays repeated-action escalation by requiring a longer zero-progress streak rather than by increasing W .

The sweep shows that the default Event Trigger is not merely an arbitrary midpoint. Making the Event Trigger more strategic increases Strategic Controller calls from 2.273 to 2.355 per step, but slightly lowers Lite-100 SR and Budgeted SR, suggesting that earlier escalation can interrupt otherwise stable local execution. Making it more reactive reduces strategic calls to 2.189 per step, but also lowers success, indicating that delayed escalation leaves stale plans and local failures active for longer. Removing periodic refresh still leaves 2.024 strategic calls per step under event-driven triggers, but causes the largest success drop, showing that event-driven triggers alone are insufficient for long-horizon recovery. Overall, Table 5 supports the default design: periodic refresh plus event-driven escalation gives the best success–cost balance among the tested settings. Because the non-default rows are lightweight one-run sensitivity checks, this table is used to inspect local trends rather than to estimate uncertainty.

All sweep settings use the same Lite-100 tasks, backend, prompts, environment interface, memory configuration, and aggregation scripts as the main runs. The default row corresponds to the fixed setting used throughout the main experiments, while each non-default setting is run once as a lightweight sensitivity check. The Strategic-calls-per-step metric counts large-model invocations made by Strategic Controller modules and divides them by the number of environment steps.

G Hierarchical Memory Implementation Details

Each SA-MB item stores a state summary, action or short action trace, reward EMA, empirical success rate, timestamp, and source tag. Repeated writes update reward EMA as $\text{rewardEMA}_i \leftarrow (1 - \eta)\text{rewardEMA}_i + \eta r_t$ with $\eta = 0.3$. Before retrieval, query and state text are lowercased, punctuation is stripped, and whitespace is collapsed. The lexical term is $L_i = |\mathcal{T}(x_t) \cap \mathcal{T}(s_i)| / |\mathcal{T}(x_t) \cup \mathcal{T}(s_i)|$, and the cosine term C_i is computed from normalized token-frequency vectors. Reward, reliability,

and recency use $R_i = (\text{clip}(\text{rewardEMA}_i, -1, 1) + 1)/2$, $P_i = \text{successes}_i / \max(1, \text{attempts}_i)$, and $\rho_i = \exp(-\text{ageHours}_i/24)$. Thus all retrieval terms are on comparable $[0, 1]$ scales before weighting. The Reactive Controller receives the top k_{MB} retrieved summaries as hints rather than executable commands.

SA-KG stores validated transitions (s_t, a_t, s_{t+1}) as state nodes and action edges. Repeated transitions update edge counts, reward EMA, and reliability. During escalation, the Strategic Controller queries the top five nearest state nodes with a Chroma vector index built from the BAAI/bge-base-en-v1.5 embedding model, keeps candidates with $C_j = 1 - \text{dist}(x_t, s_j) \geq \tau_{\text{KG}}$, and selects outgoing actions by success rate with execution count as a tie-breaker.

Fusion retrieval uses the SA-MB ranking as the primary list and injects the largest matched SA-KG action boost for each candidate action. For each retrieval call, source scores are normalized within their own candidate sets before fusion:

$$\tilde{S}_\star(c) = \frac{S_\star(c) - \min_{u \in \mathcal{C}_\star} S_\star(u)}{\max_{u \in \mathcal{C}_\star} S_\star(u) - \min_{u \in \mathcal{C}_\star} S_\star(u) + \epsilon}, \quad \star \in \{\text{MB}, \text{KG}\}. \quad (\text{A3})$$

The final fused score is

$$S_{\text{fuse}} = \lambda_{\text{MB}} \tilde{S}_{\text{MB}} + \lambda_{\text{KG}} \tilde{S}_{\text{KG}}, \quad (\text{A4})$$

where S_{KG} follows Eq. 5 and unmatched graph actions use $\tilde{S}_{\text{KG}} = 0$. After execution, successful actions update SA-MB statistics immediately. A trajectory is promoted to SA-KG only after successful completion, interpretable recovery, or Strategic Controller validation, which prevents transient errors from becoming long-term evidence.

Assets and licenses. The work uses existing research assets rather than introducing a new dataset or new foundation model. We cite the benchmark, prior agent methods, memory methods, and graph-memory work used for comparison or motivation in the main paper. The released materials will document external implementation assets, including benchmark assets, model interfaces, evaluation scripts, and figure-generation dependencies, together with corresponding license or usage terms when available.

H Additional Limitations

The proposed framework introduces more moving parts than a single-controller baseline, including Event Trigger thresholds, retrieval quality, and write filtering. These components create new failure modes: stale memory may be retrieved, local control may remain active too long, and escalation may happen either too late or too often. Strong performance on STARDOJO also does not automatically imply transfer to other games or embodied settings, especially when their action abstractions or observation formats differ greatly.

The recovery metric uses benchmark-provided execution logs to make stuck and recovery events comparable across methods. These logs are not required by the online Event Trigger, but deployment settings with only screenshots and low-level inputs would force F1–F3 detection to rely more on visual differencing, action-history patterns, and inferred progress. Such settings may make failure grading noisier, especially for subtle blocked movements or menu states that are visually unclear.

The remaining limitations separate category-specific bottlenecks from one representative failure trace, and then return to attribution limits for the multi-component design.

Task-family disparities. The lower Combat and Social success rates in Table 3 reflect category-specific bottlenecks rather than a uniform controller failure. Combat tasks require both finding the target enemy and reacting while monsters continue moving, so visual change and local navigation errors can invalidate a plan within a few steps. Social tasks often require traveling from the player’s home to another character’s home over long routes before the interaction can begin, placing heavier demands on navigation, route persistence, and subgoal planning than shorter farming or crafting tasks.

Representative combat failure case. A representative failure occurs in a rock-crab combat task (Figure A4). The agent selects the weapon and keeps the high-level objective of searching the mine



Figure A4: **Representative combat failure case.** The triptych shows a rock-crab task in the mine: the agent begins with the correct combat tool and search objective, repeatedly navigates through cluttered rocks while the target remains hard to localize, and eventually exhausts the step budget without task progress.

780 floor, but then spends most of its budget on short local moves around obstacles while task progress
 781 remains zero. Replanning events still tend to produce local navigation commands rather than a
 782 reliable search-and-engage policy. This illustrates why Combat remains difficult: the agent must
 783 couple target localization, navigation through cluttered mine geometry, and attack timing against
 784 enemies that may move or remain visually ambiguous.

785 Another limitation concerns attribution. Once multiple memory layers are available, it becomes easy
 786 to attribute gains too broadly to memory alone. We therefore separate the effects of event triggering,
 787 summarization, and long-term retrieval through explicit ablations, but finer-grained causal analysis
 788 remains future work.

789 I Responsible Use and LLM Disclosure

790 **Ethics statement.** The proposed framework is evaluated only in benchmarked game environments
 791 and does not involve human subjects, crowdsourcing, personally identifiable information, or private
 792 user data. It does not introduce a new external dataset or a new foundation model. Existing benchmark
 793 assets, model interfaces, and implementation dependencies are cited in the paper. The work does not
 794 require IRB approval because it does not involve human-subject research or interaction with private
 795 user data.

796 **Reproducibility statement.** To facilitate reproducibility, we will provide data, code, and instruc-
 797 tions with the final paper, including the controller implementation, Event Trigger and memory
 798 configuration, prompt templates, task split metadata, evaluation scripts, aggregation scripts, and
 799 instructions for reproducing the reported Lite-100 and RDR2 experiments. The final release will
 800 also record the run configurations and logs used for the reported tables. We do not release a new
 801 foundation-model checkpoint; access to the LLM backend follows the corresponding model and
 802 provider terms.

803 **Broader social impact.** Potential positive impacts include more efficient long-horizon assistive
 804 agents for complex games and software-like environments, lower evaluation cost for research on
 805 token-latency-success trade-offs, and reusable tools for studying recovery behavior in interactive
 806 agents. Potential negative impacts include misuse for game botting, automation that violates platform
 807 rules, or transfer to software-control settings where incorrect actions could cause unintended effects.
 808 Long-term memory may also preserve stale or biased state descriptions if the underlying observations
 809 or model outputs are unreliable. We mitigate release risk by presenting SPIKE as benchmark-
 810 oriented evaluation work rather than a deployment-ready automation product, and recommend human
 811 oversight, domain restrictions, logging, rate limits, and platform-rule compliance for any downstream
 812 use.

813 **LLM disclosure.** LLMs are a core method component of the work rather than a writing aid:
 814 the Strategic Controller and Reactive Controller are both implemented with large-model calls, as
 815 described in the methodology and experimental setup sections.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [Yes]

Justification: The abstract, introduction, and experimental sections consistently describe the paper as a dual controller framework for long-horizon game control with an emphasis on success, token cost, latency, and recovery; see the abstract, Secs. 1 and 3.

Guidelines:

- The answer [N/A] means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [No] or [N/A] answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.
- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: The main paper includes an explicit limitations and future work section; see Sec. 4. The appendix provides additional discussion in Sec. H.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions (e.g., independence assumptions, noiseless settings, model well-specification, asymptotic approximations only holding locally). The authors should reflect on how these assumptions might be violated in practice and what the implications would be.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs. In general, empirical results often depend on implicit assumptions, which should be articulated.
- The authors should reflect on the factors that influence the performance of the approach. For example, a facial recognition algorithm may perform poorly when image resolution is low or images are taken in low lighting. Or a speech-to-text system might not be used reliably to provide closed captions for online lectures because it fails to handle technical jargon.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that aren’t acknowledged in the paper. The authors should use their best judgment and recognize that individual actions in favor of transparency play an important role in developing norms that preserve the integrity of the community. Reviewers will be specifically instructed to not penalize honesty concerning limitations.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper is an empirical framework paper and does not introduce formal theorems; Sec. 2 keeps the Event Trigger rule as the main formal notation, while the visual change distance and remaining implementation details are provided in the appendix.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: The paper specifies the benchmark split, task distribution, model backend, controller setup, call budgets, compared baselines, metrics, repeated-run protocol, and supporting reproducibility details; see Secs. 2, 3, and B.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways. For example, if the contribution is a novel architecture, describing the architecture fully might suffice, or if the contribution is a specific model and empirical evaluation, it may be necessary to either make it possible for others to replicate the model with the same dataset, or provide access to the model. In general, releasing code and data is often one good way to accomplish this, but reproducibility can also be provided via detailed instructions for how to replicate the results, access to a hosted model (e.g., in the case of a large language model), releasing of a model checkpoint, or other means that are appropriate to the research performed.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution. For example
 - (a) If the contribution is primarily a new algorithm, the paper should make it clear how to reproduce that algorithm.
 - (b) If the contribution is primarily a new model architecture, the paper should describe the architecture clearly and fully.
 - (c) If the contribution is a new model (e.g., a large language model), then there should either be a way to access this model for reproducing the results or a way to reproduce the model (e.g., with an open-source dataset or instructions for how to construct the dataset).
 - (d) We recognize that reproducibility may be tricky in some cases, in which case authors are welcome to describe the particular way they provide for reproducibility. In the case of closed-source models, it may be that access to the model is limited in some way (e.g., to registered users), but it should be possible for other researchers to have some path to reproducing or verifying the results.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [N/A]

Justification: We will provide data, code, and instructions for the final paper.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer. Papers cannot be rejected simply for not including code, unless this is central to the contribution (e.g., for a new open-source benchmark).
- The instructions should contain the exact command and environment needed to run to reproduce the results. See the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- The authors should provide instructions on data access and preparation, including how to access the raw data, preprocessed data, intermediate data, and generated data, etc.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines. If only a subset of experiments are reproducible, they should state which ones are omitted from the script and why.
- At submission time, to preserve anonymity, the authors should release anonymized versions (if applicable).
- Providing as much information as possible in supplemental material (appended to the paper) is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: The paper specifies the benchmark split, task composition, compared methods, controller model setup, call budgets, Event Trigger and memory hyperparameter selection, evaluation metrics, and repeated-run protocol; see Secs. 2, 3, and B.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: The paper reports mean and standard deviation over three repeated runs in the quantitative tables, states that rates and ratios are computed per run before averaging, and describes the error bars in Figure 4; see Sec. 3, Sec. C, Table 2, and Table 4.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.

- The factors of variability that the error bars are capturing should be clearly stated (for example, train/test split, initialization, random drawing of some parameter, or overall run with given experimental conditions).
- The method for calculating the error bars should be explained (closed form formula, call to a library function, bootstrap, etc.)
- The assumptions made should be given (e.g., Normally distributed errors).
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- It is OK to report 1-sigma error bars, but one should state it. The authors should preferably report a 2-sigma error bar than state that they have a 96% CI, if the hypothesis of Normality of errors is not verified.
- For asymmetric distributions, the authors should be careful not to show in tables or figures symmetric error bars that would yield results that are out of range (e.g., negative error rates).
- If error bars are reported in tables or plots, the authors should explain in the text how they were calculated and reference the corresponding figures or tables in the text.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: The paper reports end-to-end wall-clock latency per step and the appendix describes the model-serving GPUs, local workstation memory and GPU, CPU worker role, worker configuration records, memory budget records, and run-time logs; see Sec. 3 and Sec. B.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper (e.g., preliminary or failed experiments that didn't make it into the paper).

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines>?

Answer: [Yes]

Justification: The work studies agent behavior in benchmarked game environments only, does not involve human subjects or personal data, and includes responsible-use, release-risk, and LLM-use statements in Sec. I.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity (e.g., if there is a special consideration due to laws or regulations in their jurisdiction).

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: The appendix includes a brief discussion of both positive applications and potential misuse risks for long-horizon automation agents; see Sec. I.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses (e.g., disinformation, generating fake profiles, surveillance), fairness considerations (e.g., deployment of technologies that could make decisions that unfairly impact specific groups), privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments. However, if there is a direct path to any negative applications, the authors should point it out. For example, it is legitimate to point out that an improvement in the quality of generative models could be used to generate Deepfakes for disinformation. On the other hand, it is not needed to point out that a generic algorithm for optimizing neural networks could enable people to train models that generate Deepfakes faster.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, harms that could arise when the technology is being used as intended but gives incorrect results, and harms following from (intentional or unintentional) misuse of the technology.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies (e.g., gated release of models, providing defenses in addition to attacks, mechanisms for monitoring misuse, mechanisms to monitor how a system learns from feedback over time, improving the efficiency and accessibility of ML).

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper documents the framework as benchmark-oriented evaluation work rather than a new high-risk pretrained model, scraped dataset, or deployment-ready automation product; see Sec. I.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model, for example by requiring that users adhere to usage guidelines or restrictions to access the model or implementing safety filters.
- Datasets that have been scraped from the Internet could pose safety risks. The authors should describe how they avoided releasing unsafe images.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: The paper cites the benchmark, compared agent methods, memory methods, and graph-memory work used for motivation or comparison, and documents external implementation assets and license terms in the appendix; see Appendix Secs. A and G.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license (e.g., CC-BY 4.0) should be included for each asset.
- For scraped data from a particular source (e.g., website), the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided. For popular datasets, paperswithcode.com/datasets has curated licenses for some datasets. Their licensing guide can help determine the license of a dataset.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset (if it has changed) should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [Yes]

Justification: The appendix documents the planned release materials, including task meta-data, configuration files, prompt templates, evaluation scripts, aggregation scripts, and release terms for the SPIKE code and configuration files; see Secs. B and I.

Guidelines:

- The answer [N/A] means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates. This includes details about training, license, limitations, etc.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets (if applicable). You can either create an anonymized URL or include an anonymized zip file.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [N/A]

Justification: The paper explicitly states that the work does not involve crowdsourcing or human subjects, so participant instructions, screenshots, and compensation details are not applicable; see Sec. I.

Guidelines:

- The answer [N/A] means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

1130 Question: Does the paper describe potential risks incurred by study participants, whether
1131 such risks were disclosed to the subjects, and whether Institutional Review Board (IRB)
1132 approvals (or an equivalent approval/review based on the requirements of your country or
1133 institution) were obtained?

1134 Answer: [N/A]

1135 Justification: The paper explicitly states that the work does not involve human subjects,
1136 crowdsourcing, or private user data, and therefore does not require IRB approval; see Sec. I.

1137 Guidelines:

- 1138 • The answer [N/A] means that the paper does not involve crowdsourcing nor research
1139 with human subjects.
- 1140 • Depending on the country in which research is conducted, IRB approval (or equivalent)
1141 may be required for any human subjects research. If you obtained IRB approval, you
1142 should clearly state this in the paper.
- 1143 • We recognize that the procedures for this may vary significantly between institutions
1144 and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the
1145 guidelines for their institution.
- 1146 • For initial submissions, do not include any information that would break anonymity (if
1147 applicable), such as the institution conducting the review.

1148 16. Declaration of LLM usage

1149 Question: Does the paper describe the usage of LLMs if it is an important, original, or
1150 non-standard component of the core methods in this research? Note that if the LLM is used
1151 only for writing, editing, or formatting purposes and does *not* impact the core methodology,
1152 scientific rigor, or originality of the research, declaration is not required.

1153 Answer: [Yes]

1154 Justification: LLMs are part of the core method: the Strategic Controller and Reactive
1155 Controller are both implemented with LLM-based components, and their roles are described
1156 in Secs. 2, 3, and I.

1157 Guidelines:

- 1158 • The answer [N/A] means that the core method development in this research does not
1159 involve LLMs as any important, original, or non-standard components.
- 1160 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not
1161 be described.